

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA0401

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

*I **G. Yashmita /192511032**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Yashmita

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Q1. Deadlock Debugging (Code Trace)

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
pthread_mutex_t R1, R2;
void *P1(void *arg)
{
    printf("P1 started\n");
    pthread_mutex_lock(&R1);
    printf("P1 locked R1\n");
    sleep(1);
    pthread_mutex_lock(&R2);
    printf("P1 locked R2\n");
    printf("P1 entered Critical Section\n");
    sleep(1);
    pthread_mutex_unlock(&R2);
    printf("P1 unlocked R2\n");
    pthread_mutex_unlock(&R1);
    printf("P1 unlocked R1\n");
    printf("P1 finished\n\n");
    pthread_exit(NULL);
}
void *P2(void *arg)
{
    printf("P2 started\n");
    // Same locking order as P1 avoids deadlock
    pthread_mutex_lock(&R1);
    printf("P2 locked R1\n");
    sleep(1);
    pthread_mutex_lock(&R2);
    printf("P2 locked R2\n");
    printf("P2 entered Critical Section\n");
    sleep(1);
```

```

    pthread_mutex_unlock(&R2);
    printf("P2 unlocked R2\n");
    pthread_mutex_unlock(&R1);
    printf("P2 unlocked R1\n");
    printf("P2 finished\n");
    pthread_exit(NULL);
}

int main()
{
    pthread_t t1, t2;
    pthread_mutex_init(&R1, NULL);
    pthread_mutex_init(&R2, NULL);
    printf("==== Deadlock Debugging =====\n");
    printf("Starting Process P1 and P2...\n\n");
    pthread_create(&t1, NULL, P1, NULL);
    pthread_join(t1, NULL);
    pthread_create(&t2, NULL, P2, NULL);
    pthread_join(t2, NULL);
    pthread_mutex_destroy(&R1);
    pthread_mutex_destroy(&R2);
    printf("\nDeadlock eliminated successfully.\n");
    printf("Both processes executed successfully.\n");
    return 0;
}

```

EXPECTED OUTPUT:

==== Deadlock Debugging =====

Starting Process P1 and P2...

P1 started

P1 locked R1

P1 locked R2

P1 entered Critical Section

P1 unlocked R2

P1 unlocked R1

P1 finished

P2 started

P2 locked R1

P2 locked R2

P2 entered Critical Section

P2 unlocked R2

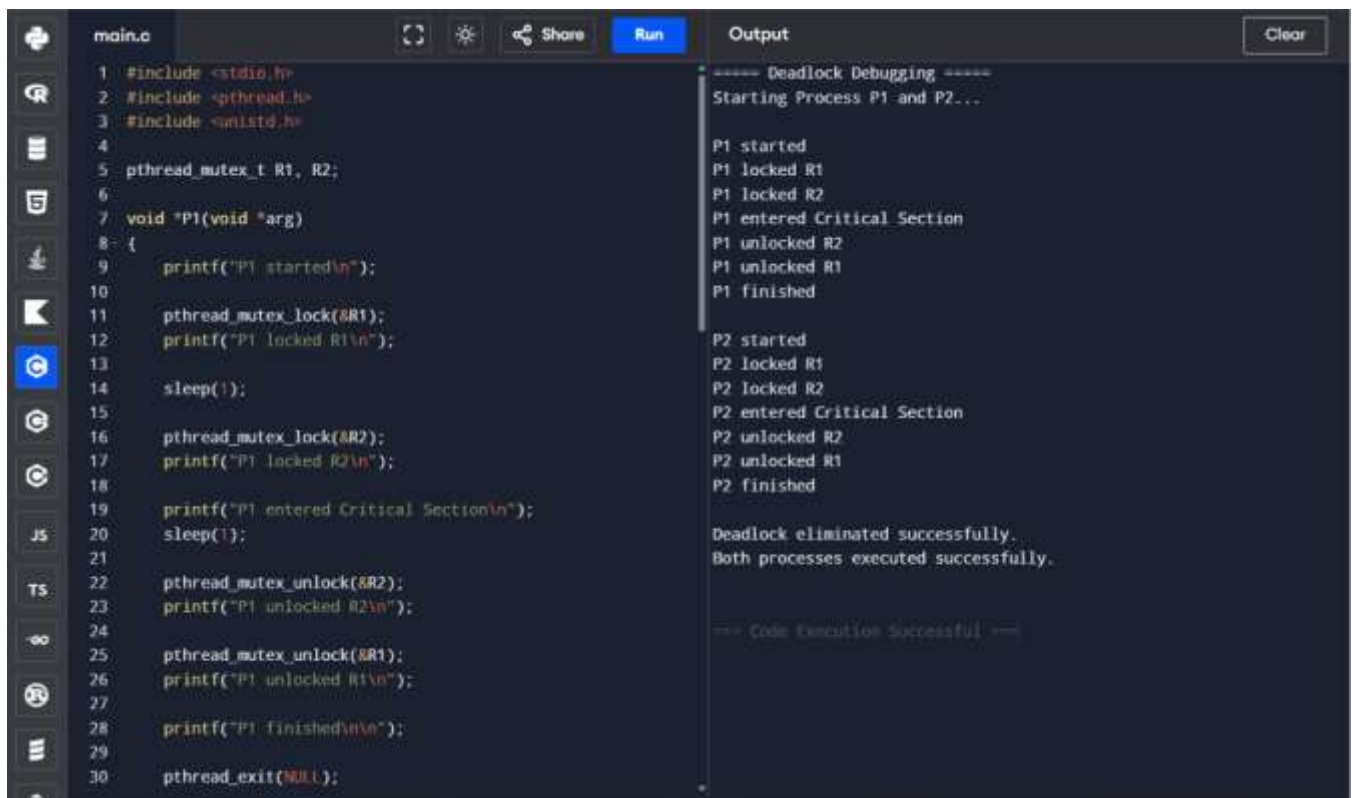
P2 unlocked R1

P2 finished

Deadlock eliminated successfully.

Both processes executed successfully.

EXECUTION:



```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4
5 pthread_mutex_t R1, R2;
6
7 void *P1(void *arg)
8 {
9     printf("P1 started\n");
10
11     pthread_mutex_lock(&R1);
12     printf("P1 locked R1\n");
13
14     sleep(1);
15
16     pthread_mutex_lock(&R2);
17     printf("P1 locked R2\n");
18
19     printf("P1 entered Critical Section\n");
20     sleep(1);
21
22     pthread_mutex_unlock(&R2);
23     printf("P1 unlocked R2\n");
24
25     pthread_mutex_unlock(&R1);
26     printf("P1 unlocked R1\n");
27
28     printf("P1 finished\n\n");
29
30     pthread_exit(NULL);
}
```

Output

```
----- Deadlock Debugging -----
Starting Process P1 and P2...

P1 started
P1 locked R1
P1 locked R2
P1 entered Critical Section
P1 unlocked R2
P1 unlocked R1
P1 finished

P2 started
P2 locked R1
P2 locked R2
P2 entered Critical Section
P2 unlocked R2
P2 unlocked R1
P2 finished

Deadlock eliminated successfully.
Both processes executed successfully.

=== Code Execution Successful ===
```

The screenshot shows a code editor with a C program in `main.c` and its output in the right-hand pane. The code implements two processes, P1 and P2, each with a critical section protected by a mutex. P1 holds R1 and R2, while P2 holds R1 and R2. The output shows the execution flow, including process startup, locking, entering critical sections, and finishing, confirming that the deadlock was eliminated successfully.

```

54
55     printf("P2 finished\n");
56
57     pthread_exit(NULL);
58 }
59
60 int main()
61 {
62     pthread_t t1, t2;
63
64     pthread_mutex_init(&R1, NULL);
65     pthread_mutex_init(&R2, NULL);
66
67     printf("==== Deadlock Debugging =====\n");
68     printf("Starting Process P1 and P2...\n\n");
69
70     pthread_create(&t1, NULL, P1, NULL);
71     pthread_join(t1, NULL);
72
73     pthread_create(&t2, NULL, P2, NULL);
74     pthread_join(t2, NULL);
75
76     pthread_mutex_destroy(&R1);
77     pthread_mutex_destroy(&R2);
78
79     printf("\nDeadlock eliminated successfully.\n");
80     printf("Both processes executed successfully.\n");
81
82     return 0;
83 }

```

Output:

```

==== Deadlock Debugging =====
Starting Process P1 and P2...

P1 started
P1 locked R1
P1 locked R2
P1 entered Critical Section
P1 unlocked R2
P1 unlocked R1
P1 finished

P2 started
P2 locked R1
P2 locked R2
P2 entered Critical Section
P2 unlocked R2
P2 unlocked R1
P2 finished

Deadlock eliminated successfully.
Both processes executed successfully.

=== Code Execution Successful ===

```

Q2. CPU Utilisation Debugging

CODE:

```

#include <stdio.h>
int main()
{
    int ready = 5;
    int waiting = 3;
    printf("CPU Scheduler Debugging\n");
    while(ready > 0)
    {
        printf("Process Running...\n");
        if(waiting > 0)
        {
            printf("One process waiting for I/O\n");
            waiting--;
            ready++;
        }
        ready--;
    }
    printf("CPU Utilisation Improved.\n");
    return 0;
}

```

EXPECTED OUTPUT:

CPU Scheduler Debugging

Process Running...

One process waiting for I/O

Process Running...

One process waiting for I/O

Process Running...

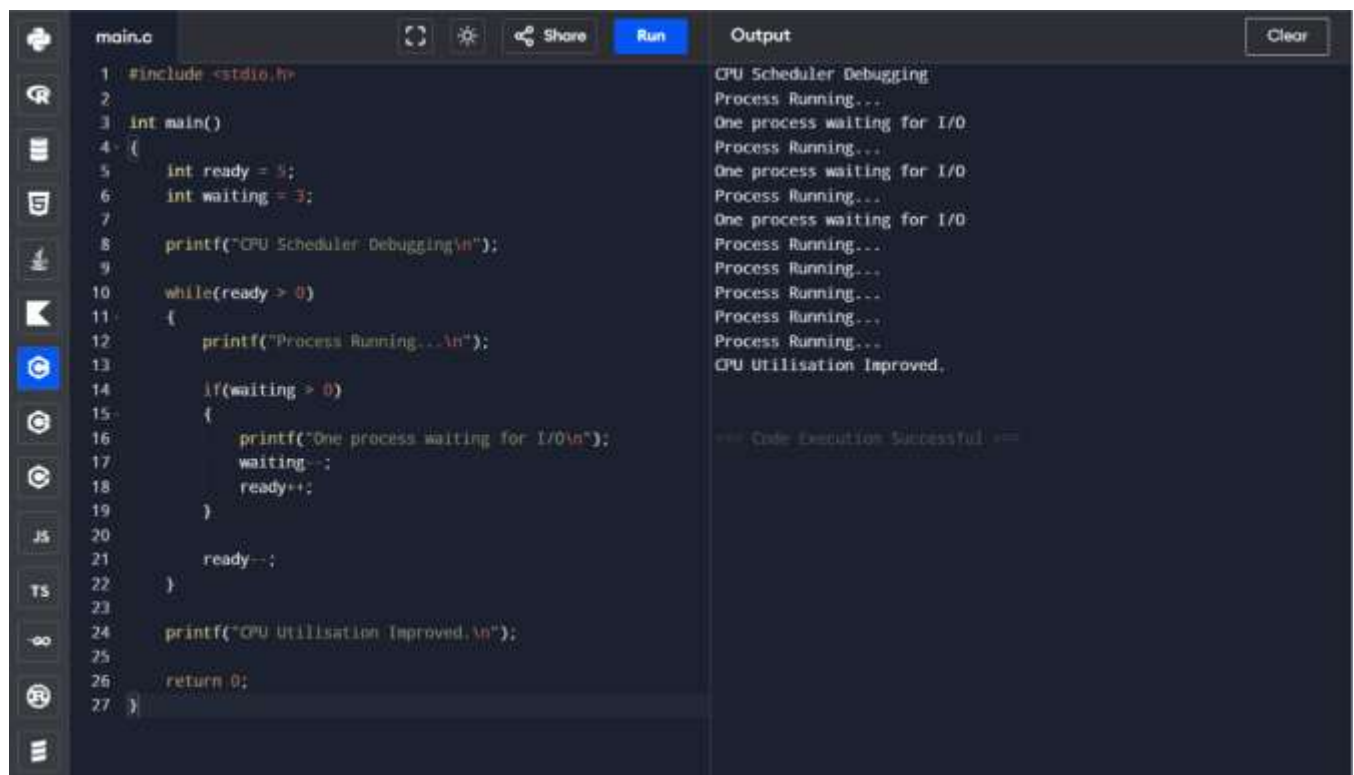
One process waiting for I/O

Process Running...

Process Running...

CPU Utilisation Improved.

EXECUTION:



The screenshot shows a code editor with a file named `main.c`. The code is a C program that simulates a CPU scheduler. It initializes `ready = 5` and `waiting = 3`. It prints "CPU Scheduler Debugging". Then it enters a `while(ready > 0)` loop. Inside the loop, it prints "Process Running...\n". If `waiting > 0`, it prints "One process waiting for I/O\n", decrements `waiting`, and increments `ready`. After the loop, it prints "CPU Utilisation Improved,\n" and returns 0. The output window on the right shows the execution of this code, displaying the same sequence of messages as the text above: "CPU Scheduler Debugging", "Process Running...", "One process waiting for I/O", "Process Running...", "One process waiting for I/O", "Process Running...", "One process waiting for I/O", "Process Running...", "Process Running...", "Process Running...", "Process Running...", "Process Running...", "Process Running...", "CPU Utilisation Improved.", and "=== Code Execution Successful ===".

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int ready = 5;
6     int waiting = 3;
7
8     printf("CPU Scheduler Debugging\n");
9
10    while(ready > 0)
11    {
12        printf("Process Running...\n");
13
14        if(waiting > 0)
15        {
16            printf("One process waiting for I/O\n");
17            waiting--;
18            ready++;
19        }
20
21        ready--;
22    }
23
24    printf("CPU Utilisation Improved,\n");
25
26    return 0;
27 }
```

Output

```
CPU Scheduler Debugging
Process Running...
One process waiting for I/O
Process Running...
One process waiting for I/O
Process Running...
One process waiting for I/O
Process Running...
Process Running...
Process Running...
Process Running...
Process Running...
Process Running...
CPU Utilisation Improved.
=== Code Execution Successful ===
```

Q3. Memory Bottleneck Debugging

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int code = 5;
```

```
    int heap = 150;
```

```
    int stack = 1;
```

```

int threads = 25;
int ram = 2048;
int total = code + heap + (stack * threads);
printf("Memory Map\n");
printf("Code = %d MB\n", code);
printf("Heap = %d MB\n", heap);
printf("Stack = %d MB\n", stack * threads);
printf("Total Memory Used = %d MB\n", total);
printf("Physical RAM = %d MB\n\n", ram);
if(total < ram)
    printf("Issue : Frequent page faults due to inefficient memory usage.\n");
else
    printf("Issue : RAM exhausted.\n");
printf("\nOptimisation 1 : Reduce heap allocation.\n");
printf("Optimisation 2 : Reduce number of threads or stack size.\n");
return 0;
}

```

EXPECTED OUTPUT:

Memory Map

Code = 5 MB

Heap = 150 MB

Stack = 25 MB

Total Memory Used = 180 MB

Physical RAM = 2048 MB

Issue : Frequent page faults due to inefficient memory usage.

Optimisation 1 : Reduce heap allocation.

Optimisation 2 : Reduce number of threads or stack size.

EXECUTION:

The screenshot shows a code editor with a C program in a file named `main.c`. The program calculates memory usage and compares it to physical RAM. The output window shows the results of the program's execution.

```

2
3 int main()
4 {
5     int code = 5;
6     int heap = 150;
7     int stack = 1;
8     int threads = 25;
9     int ram = 2048;
10
11     int total = code + heap + (stack * threads);
12
13     printf("Memory Map\n");
14     printf("Code = %d MB\n", code);
15     printf("Heap = %d MB\n", heap);
16     printf("Stack = %d MB\n", stack * threads);
17     printf("Total Memory Used = %d MB\n", total);
18     printf("Physical RAM = %d MB\n", ram);
19
20     if(total < ram)
21         printf("Issue : Frequent page faults due to\n inefficient memory usage.\n");
22     else
23         printf("Issue : RAM exhausted.\n");
24
25     printf("\nOptimisation 1 : Reduce heap allocation.\n");
26     printf("Optimisation 2 : Reduce number of threads or\n stack size.\n");
27
28     return 0;
29 }

```

Output:

```

Memory Map
Code = 5 MB
Heap = 150 MB
Stack = 25 MB
Total Memory Used = 180 MB
Physical RAM = 2048 MB

Issue : Frequent page faults due to inefficient memory usage.

Optimisation 1 : Reduce heap allocation.
Optimisation 2 : Reduce number of threads or stack size.

=== Code Execution Successful ===

```

Q4. Starvation Debugging

CODE:

```

#include <stdio.h>
int main()
{
    int Q1 = 6;
    int Q2 = 3;
    int P5 = 0;

    while(Q1 > 0 || Q2 > 0)
    {
        if(Q1 > 0)
            Q1--;
        else if(Q2 > 0)
            Q2--;
    }
    if(Q1 == 0 && Q2 == 0)
        P5 = 1;
    if(P5)
        printf("P5 gets CPU after Aging.\n");
    else
        printf("P5 is Starved.\n");
    printf("Root Cause : Continuous execution of higher priority queues.\n");
    printf("Optimisation : Apply Aging to increase P5 priority.\n");
    return 0;
}

```

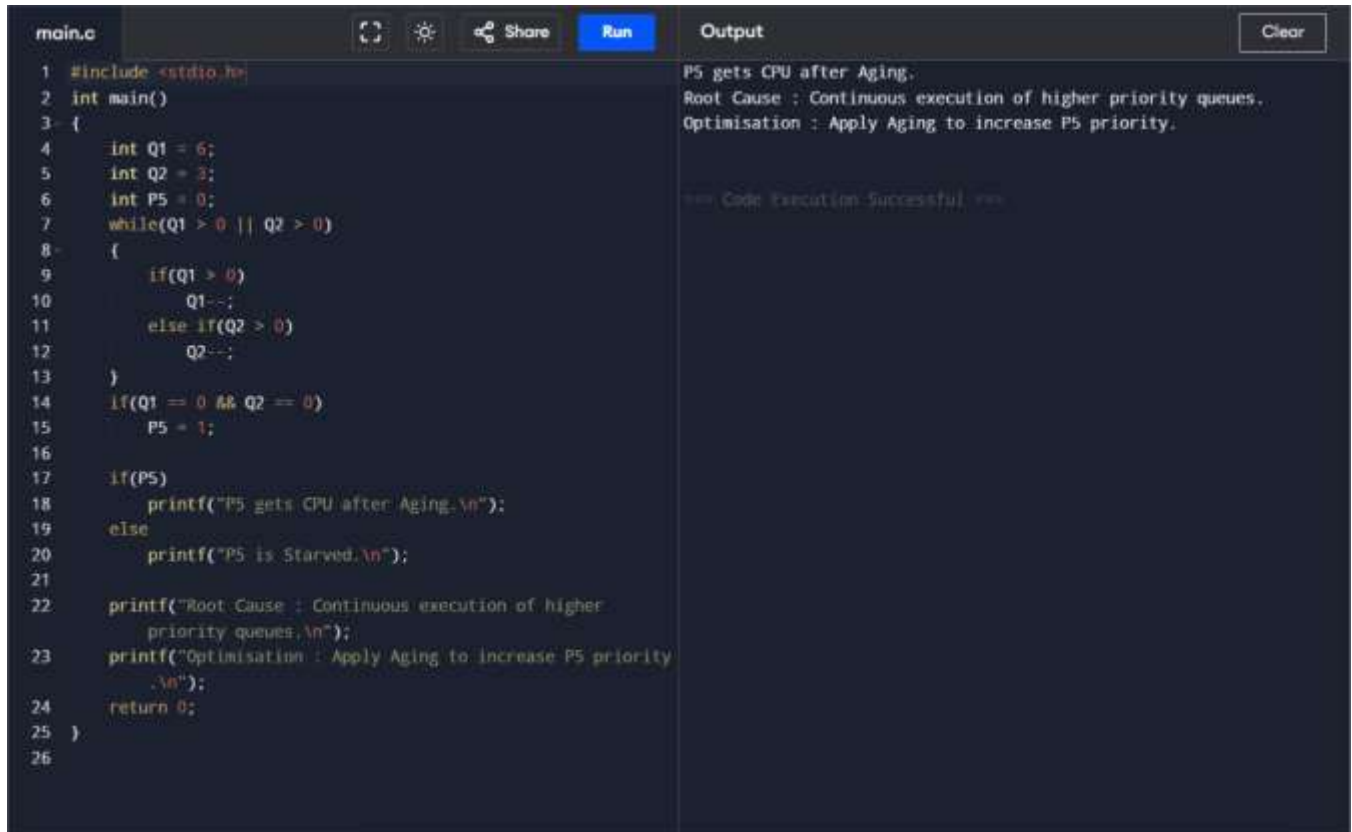
EXPECTED OUTPUT:

P5 gets CPU after Aging.

Root Cause : Continuous execution of higher priority queues.

Optimisation : Apply Aging to increase P5 priority.

EXECUTION:



The screenshot shows a C code editor with a file named 'main.c'. The code implements a scheduling algorithm with two queues, Q1 and Q2, and a process P5. Q1 starts with a value of 6, and Q2 starts with 3. P5 starts with a priority of 0. The code enters a while loop that continues as long as either Q1 or Q2 is greater than 0. Inside the loop, it checks if Q1 is greater than 0; if so, it decrements Q1. Otherwise, it checks if Q2 is greater than 0 and decrements it. After the loop, it checks if both Q1 and Q2 are 0. If so, it increments P5's priority by 1. Then, it checks if P5's priority is greater than 0. If so, it prints 'P5 gets CPU after Aging.\n'. Otherwise, it prints 'P5 is Starved.\n'. Finally, it prints the root cause and the optimization, and returns 0.

```
1 #include <stdio.h>
2 int main()
3 {
4     int Q1 = 6;
5     int Q2 = 3;
6     int P5 = 0;
7     while(Q1 > 0 || Q2 > 0)
8     {
9         if(Q1 > 0)
10            Q1--;
11        else if(Q2 > 0)
12            Q2--;
13    }
14    if(Q1 == 0 && Q2 == 0)
15        P5 = 1;
16
17    if(P5)
18        printf("P5 gets CPU after Aging.\n");
19    else
20        printf("P5 is Starved.\n");
21
22    printf("Root Cause : Continuous execution of higher
23    priority queues.\n");
24    printf("Optimisation : Apply Aging to increase P5 priority
25    .\n");
26    return 0;
27 }
```

The output window shows the following text:

```
P5 gets CPU after Aging.
Root Cause : Continuous execution of higher priority queues.
Optimisation : Apply Aging to increase P5 priority.

== Code Execution Successful ==
```

Q5. Context-Switch Overhead Debugging

CODE:

```
#include <stdio.h>
int main()
{
    int threads = 10000;    // Total threads
    int burst = 2;          // Burst time (ms)
    int quantum = 1;        // Initial time quantum
    int ioWait = 1;         // High I/O wait (1 = Yes)
    int switchesBefore = 0, switchesAfter = 0;
    // Calculate context switches before optimization
    for(int i = 0; i < threads; i++)
    {
        if(burst > quantum)
            switchesBefore += burst / quantum;

        if(ioWait)
            switchesBefore++;
    }
}
```

```

    }
    // Optimization 1: Increase time quantum
    quantum = 4;
    // Calculate context switches after optimization
    for(int i = 0; i < threads; i++)
    {
        if(burst > quantum)
            switchesAfter += burst / quantum;
        else
            switchesAfter++;
        if(ioWait)
            switchesAfter++;
    }
    printf("Before Optimization\n");
    printf("-----\n");
    printf("Average Burst Time : %d ms\n", burst);
    printf("Time Quantum      : 1 ms\n");
    printf("Context Switches   : %d\n\n", switchesBefore);
    printf("Reason:\n");
    printf("Small time quantum and frequent I/O waits cause excessive context switching.\n\n");
    printf("After Optimization\n");
    printf("-----\n");
    printf("New Time Quantum   : %d ms\n", quantum);
    printf("Context Switches   : %d\n\n", switchesAfter);
    printf("Optimisation 1 : Increase time quantum.\n");
    printf("Optimisation 2 : Improve I/O scheduling (reduce unnecessary blocking).\n");
    return 0;
}

```

EXPECTED OUTPUT:

Before Optimization

Average Burst Time : 2 ms

Time Quantum : 1 ms

Context Switches : 30000

Reason:

Small time quantum and frequent I/O waits cause excessive context switching.

After Optimization

New Time Quantum : 4 ms

Context Switches : 20000

Optimisation 1 : Increase time quantum.

Optimisation 2 : Improve I/O scheduling (reduce unnecessary blocking).

EXECUTION:

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int threads = 10000;    // Total threads
6     int burst = 2;          // Burst time (ms)
7     int quantum = 1;        // Initial time quantum
8     int iowait = 1;         // High I/O wait (1 = Yes)
9
10    int switchesBefore = 0, switchesAfter = 0;
11
12    // Calculate context switches before optimization
13    for(int i = 0; i < threads; i++)
14    {
15        if(burst > quantum)
16            switchesBefore += burst / quantum;
17
18        if(iowait)
19            switchesBefore++;
20    }
21
22    // Optimization 1: Increase time quantum
23    quantum = 4;
24
25    // Calculate context switches after optimization
26    for(int i = 0; i < threads; i++)
27    {
28        if(burst > quantum)
29            switchesAfter += burst / quantum;
30        else
```

Before Optimization

Average Burst Time : 2 ms
Time Quantum : 1 ms
Context Switches : 30000

Reason:
Small time quantum and frequent I/O waits cause excessive context switching.

After Optimization

New Time Quantum : 4 ms
Context Switches : 20000

Optimisation 1 : Increase time quantum.
Optimisation 2 : Improve I/O scheduling (reduce unnecessary blocking).

--- Code Execution Successful ---

```
31     switchesAfter++;
32
33     if(iowait)
34         switchesAfter++;
35 }
36
37 printf("Before Optimization\n");
38 printf("-----\n");
39 printf("Average Burst Time : %d ms\n", burst);
40 printf("Time Quantum      : 1 ms\n");
41 printf("Context Switches   : %d\n", switchesBefore);
42
43 printf("Reason:\n");
44 printf("Small time quantum and frequent I/O waits cause\nexcessive context switching.\n");
45
46 printf("After Optimization\n");
47 printf("-----\n");
48 printf("New Time Quantum    : 4 ms\n");
49 printf("Context Switches    : 2000\n");
50
51 printf("Optimisation 1 : Increase time quantum.\n");
52 printf("Optimisation 2 : Improve I/O scheduling (reduce unnecessary blocking).\n");
53
54 return 0;
55 }
```

Average Burst Time : 2 ms
Time Quantum : 1 ms
Context Switches : 30000

Reason:
Small time quantum and frequent I/O waits cause excessive context switching.

After Optimization

New Time Quantum : 4 ms
Context Switches : 20000

Optimisation 1 : Increase time quantum.
Optimisation 2 : Improve I/O scheduling (reduce unnecessary blocking).

=== Code Execution Successful ===